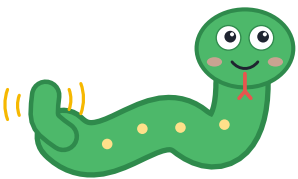
Lecture 7

Part 3: Functions and Dictionaries

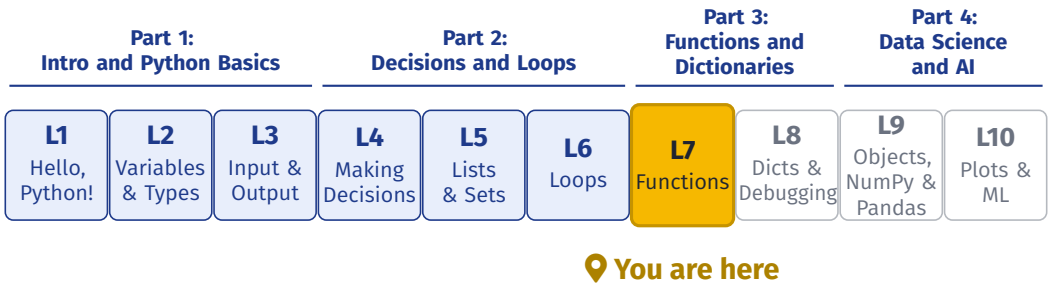
Functions

Name a recipe once, use it forever



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

Looking back: lectures 5 and 6

✓ Lecture 5: Lists & Sets

- A list `[ ]` keeps order: `append`, `insert`, `remove`, `pop`, `sort`
- **`in`**, `len`, `sum`, `max`, `min` work on any list
- A set has no duplicates: `&` both, `|` all, `-` only mine
- Positions start at 0, `-1` is the last, `[start:stop:step]` slices

✓ Lecture 6: Loops

- **`while`** repeats as long as a condition is True
- **`for`** visits every item of a list, string or `range`
- **`break`** is the emergency exit
- Loop recipes: total, average, count, max, search

Warm-up: lectures 5 and 6 in one program

```
coffees = [2, 0, 3, 1, 4, 2, 2]
total = 0
for c in coffees:
    total = total + c
print("Total:", total)
print("Days:", len(coffees))
print("Average:", total / 7)
print("Weekend:", coffees[-2:])
```

Warm-up: lectures 5 and 6 in one program

```
coffees = [2, 0, 3, 1, 4, 2, 2]
total = 0
for c in coffees:
    total = total + c
print("Total:", total)
print("Days:", len(coffees))
print("Average:", total / 7)
print("Weekend:", coffees[-2:])
```

OUTPUT

```
Total: 14
Days: 7
Average: 2.0
Weekend: [2, 2]
```

Quick check: predict the output

```
songs = ["a", "b", "c", "d", "e"]
for i in range(1, 5, 2):
    print(songs[i])
print(set([1, 2, 2, 3]) & {2, 3, 4})
```

Quick check: predict the output

```
songs = ["a", "b", "c", "d", "e"]
for i in range(1, 5, 2):
    print(songs[i])
print(set([1, 2, 2, 3]) & {2, 3, 4})
```

OUTPUT

```
b
d
{2, 3}
```

Quick check: predict the output

```
songs = ["a", "b", "c", "d", "e"]
for i in range(1, 5, 2):
    print(songs[i])
print(set([1, 2, 2, 3]) & {2, 3, 4})
```

OUTPUT

```
b
d
{2, 3}
```

- `range(1, 5, 2)` gives 1, 3: positions, not values
- `songs[1]` is the second song, `"b"`
- The set drops the extra 2, then `&` keeps what both share

“The usual, please”

Monday at the café

“Large coffee, oat milk, one sugar, extra hot, to go.”

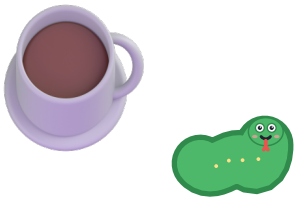
“The usual, please”

Monday at the café

“Large coffee, oat milk, one sugar, extra hot, to go.”

Friday at the café

“The usual!” (two words = five steps)



“The usual, please”

Monday at the café

“Large coffee, oat milk, one sugar, extra hot, to go.”

Friday at the café

“The usual!” (two words = five steps)





A function = a name for a list of steps.

Today's plan

◎ By the end of today you can...

- Write your own functions and call them
- Send values in and get an answer back
- Know where a variable lives (inside or outside)



Today's outline

1 **Your first functions**
`def`, calling, parameters, a function with a loop inside

2 **Getting answers back: return**
`return` vs `print`, default values, building with small bricks

3 **Where variables live: scope**
Local vs global, and why a variable can "disappear"



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Big programs are built from small, named pieces
- Write it once, use it everywhere. Fix a bug in one place
- Teams split work by functions

Where you will see it

- Every library you will use: functions others wrote
- `print`, `len`, `max` are functions too
- Apps: “pay”, “send”, “share” buttons





Topic 1

Your first functions

Lecture 7 • Functions



TODAY

1. **Your first functions**
2. Getting answers back: return
3. Where variables live: scope

Surprise: you already use functions

```
grades = [85, 92, 58]
print(len(grades))
print(max(grades))
print(type(3.5))
```

- **print**, len, max, type…
- A name + ( ) = “do it now”
- Today: write **your own**



Surprise: you already use functions

```
grades = [85, 92, 58]
print(len(grades))
print(max(grades))
print(type(3.5))
```

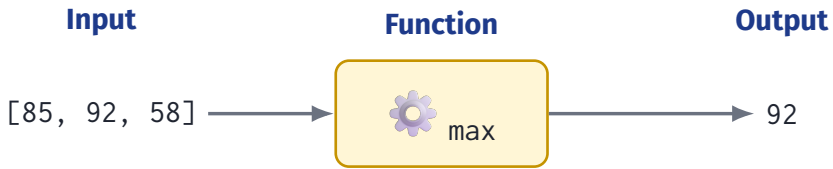
OUTPUT

```
3
92
<class 'float'>
```

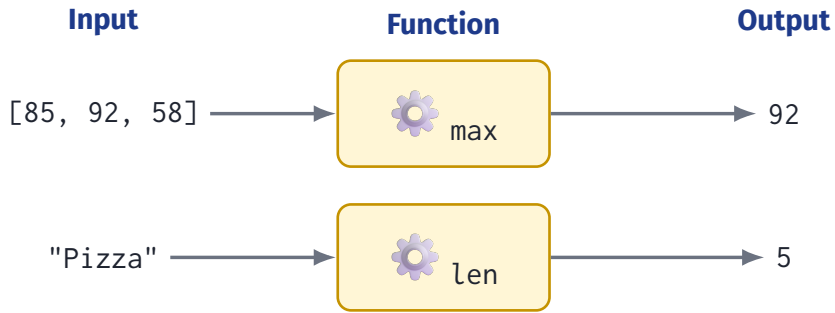
- **print**, len, max, type…
- A name + ( ) = “do it now”
- Today: write **your own**



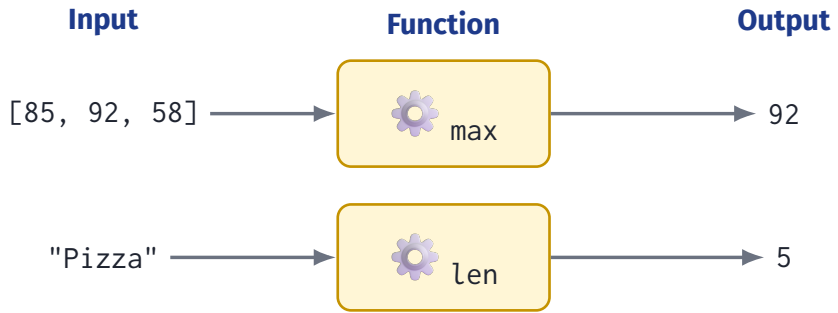
A function is a little machine



A function is a little machine



A function is a little machine



You don’t need to know what’s inside. Just what goes in and what comes out.

Your very first function

```
def say_hi():
    print("Hi there!")
    print("Welcome to class.")

say_hi()
say_hi()
```



- **`def`** = “define”: teach a new word
- `say_hi()` = use it (a **call**)

Your very first function

```
def say_hi():
    print("Hi there!")
    print("Welcome to class.")

say_hi()
say_hi()
```

OUTPUT

```
Hi there!
Welcome to class.
Hi there!
Welcome to class.
```



- `def` = “define”: teach a new word
- `say_hi()` = use it (a **call**)

Anatomy of a function

```
def greet(name):
    print("Hi", name)
```

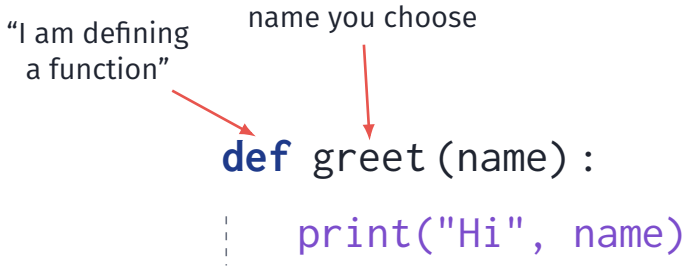
Anatomy of a function

“I am defining a function”

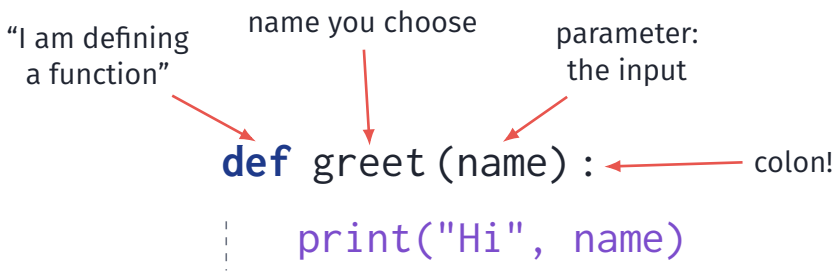


```
def greet(name):
    print("Hi", name)
```

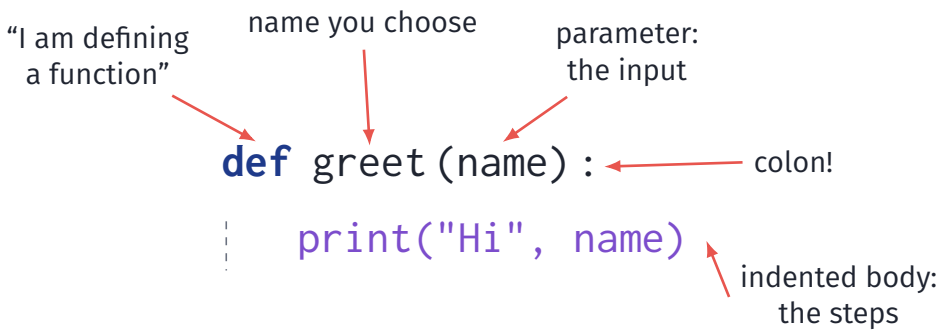
Anatomy of a function



Anatomy of a function



Anatomy of a function



Same colon + indent rule as `if`, `for` and `while`.

Parameters: fill in the blank

```python
def greet(name):
    print(f"Hi {name}, welcome!")

greet("Dana")
greet("Yossi")
```

Like a form

“Name: _____”
Each call fills in the blank.

Parameters: fill in the blank

```python
def greet(name):
    print(f"Hi {name}, welcome!")

greet("Dana")
greet("Yossi")
```

OUTPUT

```
Hi Dana, welcome!
Hi Yossi, welcome!
```

Like a form

“Name: _____”
Each call fills in the blank.

Parameters: fill in the blank

```python
def greet(name):
    print(f"Hi {name}, welcome!")

greet("Dana")
greet("Yossi")
```

OUTPUT

```
Hi Dana, welcome!
Hi Yossi, welcome!
```

Like a form

“Name: _____”
Each call fills in the blank.

`name` is the **parameter**.
`"Dana"` is the **argument**.

Two parameters

```
def order_pizza(size, topping):
    print(f"One {size} pizza with {topping}, coming up!")

order_pizza("large", "olives")
order_pizza("small", "mushrooms")
```



Two parameters

```python
def order_pizza(size, topping):
    print(f"One {size} pizza with {topping}, coming up!")

order_pizza("large", "olives")
order_pizza("small", "mushrooms")
```

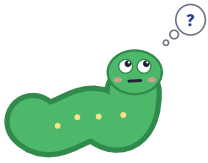
OUTPUT

```
One large pizza with olives, coming up!
One small pizza with mushrooms, coming up!
```



Order matters!

```
order_pizza("olives", "large")
```

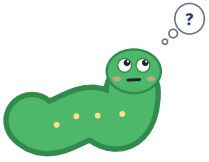


Order matters!

```
order_pizza("olives", "large")
```

OUTPUT

```
One olives pizza with large, coming up!
```

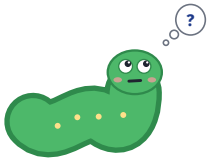


Order matters!

```
order_pizza("olives", "large")
```

OUTPUT

```
One olives pizza with large, coming up!
```



First argument → first parameter.
Second argument → second parameter.
The literal intern strikes again.

A function can hold a loop

```
def cheer(team, times):
    for i in range(times):
        print(f"Go {team}!")

cheer("Maccabi", 3)
```

Anything you learned so far can go inside a function: **if**, **for**, lists…



A function can hold a loop

```python
def cheer(team, times):
    for i in range(times):
        print(f"Go {team}!")

cheer("Maccabi", 3)
```

OUTPUT

```
Go Maccabi!
Go Maccabi!
Go Maccabi!
```



Anything you learned so far can go inside a function: **`if`**, **`for`**, lists…

Try it!

Try it in Colab notebooks/L07/L07_01_def_and_call.ipynb

1. Write `say_bye()` that prints two goodbye lines. Call it
2. Write `greet(name)` and greet three classmates
3. Write `order_drink(size, drink)` and order two drinks





Predict the output

Predict the output!

```
def hello(name):
    print("Hi", name)

print("Start")
hello("Noa")
hello("Omer")
print("End")
```

Predict the output

Predict the output!

```
def hello(name):
    print("Hi", name)

print("Start")
hello("Noa")
hello("Omer")
print("End")
```

Answer

```
Start
Hi Noa
Hi Omer
End
```

`def` only *teaches* the steps. They run at each call.

Topic 2

Getting answers back: return

Lecture 7 • Functions



TODAY

1 Your first functions

2 **Getting answers back: return**

3 Where variables live: scope

print shouts, return hands you a note



`print`

Your friend shouts the bill split across the café. You hear it, but you can't *use* it.



`return`

Your friend hands you a note with the number. Keep it, add to it, pass it on.

Your first return

```python
def pizza_share(price, people):
    return price / people

my_part = pizza_share(96, 4)
print(my_part)
print(f"Everyone pays {my_part} ILS")
```



Your first return

```python
def pizza_share(price, people):
    return price / people

my_part = pizza_share(96, 4)
print(my_part)
print(f"Everyone pays {my_part} ILS")
```

OUTPUT

```
24.0
Everyone pays 24.0 ILS
```



Your first return

```python
def pizza_share(price, people):
    return price / people

my_part = pizza_share(96, 4)
print(my_part)
print(f"Everyone pays {my_part} ILS")
```

OUTPUT

```
24.0
Everyone pays 24.0 ILS
```



`return` sends the answer back. The call `pizza_share(96, 4)` **becomes** `24.0`.

return vs print: side by side

```
def double_return(x):
    return x * 2

def double_print(x):
    print(x * 2)

a = double_return(5)
b = double_print(5)
print(a, b)
```

return vs print: side by side

```
def double_return(x):
    return x * 2

def double_print(x):
    print(x * 2)

a = double_return(5)
b = double_print(5)
print(a, b)
```

OUTPUT

```
10
10 None
```

return vs print: side by side

```
def double_return(x):
    return x * 2

def double_print(x):
    print(x * 2)

a = double_return(5)
b = double_print(5)
print(a, b)
```

OUTPUT

```
10
10 None
```

First `10`: printed *inside* `double_print`.

`b` got nothing back. Python calls "nothing" `None`.

Use the answer like any value

```python
def with_tip(bill):
    return bill * 1.12

dinner = with_tip(200)
drinks = with_tip(50)
print(f"Total: {dinner + drinks:.2f} ILS")
```



Use the answer like any value

```python
def with_tip(bill):
    return bill * 1.12

dinner = with_tip(200)
drinks = with_tip(50)
print(f"Total: {dinner + drinks:.2f} ILS")
```

OUTPUT

```
Total: 280.00 ILS
```



Use the answer like any value

```python
def with_tip(bill):
    return bill * 1.12

dinner = with_tip(200)
drinks = with_tip(50)
print(f"Total: {dinner + drinks:.2f} ILS")
```

OUTPUT

```
Total: 280.00 ILS
```



Returned values can be stored, added, printed…

A 12% tip: multiply by 1.12.

return ends the function

```
def passed(grade):
    if grade >= 60:
        return "Pass"
    return "Fail"

print(passed(75))
print(passed(42))
```



return ends the function

```python
def passed(grade):
    if grade >= 60:
        return "Pass"
    return "Fail"

print(passed(75))
print(passed(42))
```

OUTPUT

```
Pass
Fail
```



return ends the function

```
def passed(grade):
    if grade >= 60:
        return "Pass"
    return "Fail"

print(passed(75))
print(passed(42))
```

OUTPUT

```
Pass
Fail
```



At `return` the function stops. For 75: `"Pass"` is returned, the last line never runs.

Default values: “the usual”

```python
def make_coffee(size, sugar=0):
    print(f"A {size} coffee with {sugar} sugars")

make_coffee("large")
make_coffee("small", 2)
```



Default values: “the usual”

```python
def make_coffee(size, sugar=0):
    print(f"A {size} coffee with {sugar} sugars")

make_coffee("large")
make_coffee("small", 2)
```

OUTPUT

```
A large coffee with 0 sugars
A small coffee with 2 sugars
```



Default values: “the usual”

```python
def make_coffee(size, sugar=0):
    print(f"A {size} coffee with {sugar} sugars")

make_coffee("large")
make_coffee("small", 2)
```

OUTPUT

```
A large coffee with 0 sugars
A small coffee with 2 sugars
```

No sugar given? Use `0`.

Defaults go **after** the others.



Write it once, use it on anything

```
def average(numbers):
    total = 0
    for num in numbers:
        total = total + num
    return total / len(numbers)

print(average([85, 92, 58, 74]))
print(average([4200, 8100, 12600]))
```



Write it once, use it on anything

```
def average(numbers):
    total = 0
    for num in numbers:
        total = total + num
    return total / len(numbers)

print(average([85, 92, 58, 74]))
print(average([4200, 8100, 12600]))
```

OUTPUT

```
77.25
8300.0
```



Write it once, use it on anything

```
def average(numbers):
    total = 0
    for num in numbers:
        total = total + num
    return total / len(numbers)

print(average([85, 92, 58, 74]))
print(average([4200, 8100, 12600]))
```

OUTPUT

```
77.25
8300.0
```



The loop recipe from Lecture 6, now with a name. Any list works.

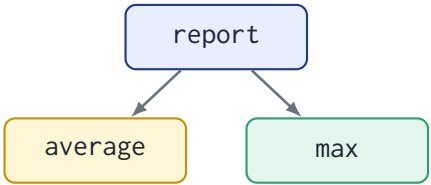
Big programs from small bricks

`report`

- Each brick does one job
- Test each brick alone
- Fix a bug once, fixed everywhere



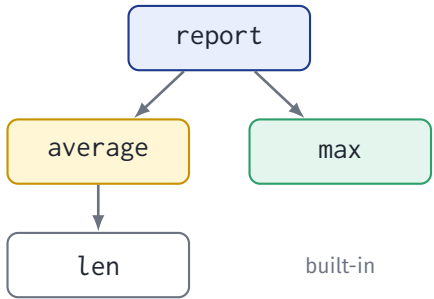
Big programs from small bricks



- Each brick does one job
- Test each brick alone
- Fix a bug once, fixed everywhere



Big programs from small bricks



- Each brick does one job
- Test each brick alone
- Fix a bug once, fixed everywhere



The bricks, put together

```
def report(name, grades):
    avg = average(grades)
    best = max(grades)
    print(f"{name}: average {avg:.1f}, best {best}")

report("Noa", [85, 92, 58, 74])
report("Omer", [55, 61, 90])
```

Uses `average` from the previous slide. A function can call other functions!

The bricks, put together

```
def report(name, grades):
    avg = average(grades)
    best = max(grades)
    print(f"{name}: average {avg:.1f}, best {best}")

report("Noa", [85, 92, 58, 74])
report("Omer", [55, 61, 90])
```

OUTPUT

```
Noa: average 77.2, best 92
Omer: average 68.7, best 90
```

Uses `average` from the previous slide. A function can call other functions!

Built-in functions vs. your own

	Built-in	Your own
Examples	`print`, `len`, `max`	`greet`, `average`
Who wrote it?	The Python team	You, with `def`
Ready to use?	Always	After the `def` runs
How to call	`len(grades)`	`average(grades)`





Once defined, yours work just like Python's.

Try it!

Try it in Colab notebooks/L07/L07_02_return.ipynb

1. Write `minutes_to_hours(minutes)` that **returns** hours
2. Write `passed(grade)` and test it on 59, 60 and 100
3. Write `greet(name, greeting="Hello")`. Call it both ways





Predict the output

Predict the output!

```
def add(a, b):
    print(a + b)

total = add(2, 3)
print(total)
```

Predict the output

Predict the output!

```
def add(a, b):
    print(a + b)

total = add(2, 3)
print(total)
```

Answer

5
None

`add` *prints* 5 but returns nothing, so `total` is `None`.

Topic 3

Where variables live: scope

Lecture 7 • Functions



TODAY

1 Your first functions

2 Getting answers back: return

3 Where variables live: scope

What happens in the function…

In the kitchen

The chef scribbles notes while cooking, then throws them away. You only get the dish.

What happens in the function…

In the kitchen

The chef scribbles notes while cooking, then throws them away. You only get the dish.



- Variables made inside a function are **local**
- They vanish when the function ends
- Only the `return` value comes out

...stays in the function

```
def make_tea():
    kettle = "hot"
    print(f"The kettle is {kettle}")

make_tea()
print(kettle)
```

...stays in the function

```python
def make_tea():
    kettle = "hot"
    print(f"The kettle is {kettle}")

make_tea()
print(kettle)
```

OUTPUT

```
The kettle is hot
```

ERROR

```
NameError: name 'kettle'
is not defined
```

...stays in the function

```python
def make_tea():
    kettle = "hot"
    print(f"The kettle is {kettle}")

make_tea()
print(kettle)
```

OUTPUT

```
The kettle is hot
```

ERROR

```
NameError: name 'kettle'
is not defined
```

`kettle` lived only inside `make_tea`.
Outside, Python has never heard of it.



Inside can look outside

```
city = "Tel Aviv"

def where_am_i():
    print(f"I am in {city}")

where_am_i()
```



Inside can look outside

```
city = "Tel Aviv"

def where_am_i():
    print(f"I am in {city}")

where_am_i()
```

OUTPUT

```
I am in Tel Aviv
```



Inside can look outside

```
city = "Tel Aviv"

def where_am_i():
    print(f"I am in {city}")

where_am_i()
```

OUTPUT

```
I am in Tel Aviv
```



`city` is **global**: made outside every function.
Functions can read it.

Same name, two different boxes

```
mood = "sleepy"

def coffee_break():
    mood = "awake"
    print("Inside:", mood)

coffee_break()
print("Outside:", mood)
```

Same name, two different boxes

```
mood = "sleepy"

def coffee_break():
    mood = "awake"
    print("Inside:", mood)

coffee_break()
print("Outside:", mood)
```

OUTPUT

```
Inside: awake
Outside: sleepy
```

Same name, two different boxes

```
mood = "sleepy"

def coffee_break():
    mood = "awake"
    print("Inside:", mood)

coffee_break()
print("Outside:", mood)
```

OUTPUT

```
Inside: awake
Outside: sleepy
```

Main program
`mood = "sleepy"`

Inside `coffee_break`
`mood = "awake"`

erased when the function ends

Parameters are local too

```
def square(n):
    n = n * n
    return n

n = 5
result = square(n)
print(result)
print(n)
```



Parameters are local too

```
def square(n):
    n = n * n
    return n

n = 5
result = square(n)
print(result)
print(n)
```

OUTPUT

```
25
5
```



Parameters are local too

```
def square(n):
    n = n * n
    return n

n = 5
result = square(n)
print(result)
print(n)
```

OUTPUT

```
25
5
```

The function changed *its own* `n`.
The outside `n` is still `5`.



The golden rule

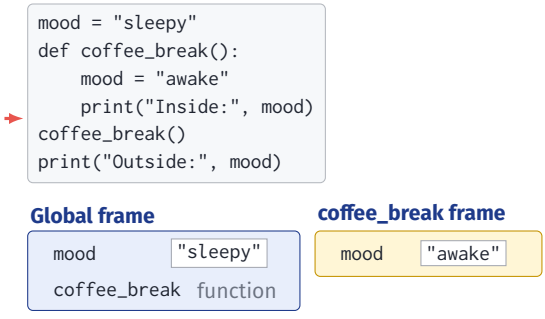


Data goes in through parameters and comes out through return.

Watch it happen: Python Tutor

1. Open **pythontutor.com**
2. Paste your code
3. Click the button to visualize
4. Step forward, line by line

You see each function's own box of variables appear and vanish.



Drawing of what Python Tutor shows, mid-run

Try it!

Try it in Colab notebooks/L07/L07_03_scope.ipynb

1. Run the `make_tea` example. Read the error
2. Fix it: `return` the kettle, store it outside
3. Paste the `coffee_break` example into Python Tutor





Predict the output

Predict the output!

```
def my_f():
    i = 5
    print("local", i)

i = 3
print("global", i)
my_f()
print("global", i)
```

Predict the output

Predict the output!

```
def my_f():
    i = 5
    print("local", i)

i = 3
print("global", i)
my_f()
print("global", i)
```

Answer

```
global 3
local 5
global 3
```

The `i` inside `my_f` is a different box.
The outside `i` stays 3.

Bug Zoo: calling too early

BUG ZOO

```
greet("Dana")

def greet(name):
    print(f"Hi {name}!")
```

ERROR

```
NameError: name 'greet' is not defined
```

Use it before you teach it

Python reads top to bottom. At line 1, `greet` does not exist yet.

Fix: put the `def` first, then the call.



Bug Zoo: a missing argument

BUG ZOO

```
def greet(name):
    print(f"Hi {name}!")

greet()
```

ERROR

```
TypeError: greet() missing 1
required positional argument:
'name'
```

An empty blank

`greet` needs a name, but the call gave none.

Fix: `greet("Dana")`, or give a default:

```
def greet(name="friend"):
```

Bug Zoo: the forgotten return

BUG ZOO

```
def add_tip(bill):
    total = bill * 1.12

dinner = add_tip(100)
print(dinner + 20)
```

ERROR

```
TypeError: unsupported operand
type(s) for +: 'NoneType' and
'int'
```

No return = None

`total` stays inside. `dinner` gets `None`, and `None + 20` makes no sense.

Fix: add `return total` as the last line.

AI Corner: test your function

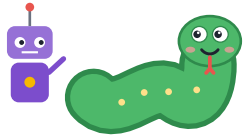
AI CORNER

A good prompt

“Here is my Python function: [paste]. In simple words: what goes in, what comes out? Give 3 test calls with their answers.”

Double-check

Run the 3 test calls yourself. Same answers as the AI promised?



Recap

✓ Today you learned

- `def` names a list of steps, a call runs them
- Parameters bring values in, `return` sends an answer out
- Variables made inside a function stay inside



Next time: dictionaries and debugging

- A phone book in Python: look up by name
- Tables of data: lists inside lists
- Reading errors like a detective, and catching them with `try`
- Coding together with AI, the smart way





3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md